

整合練習 5：BDC Program 實作

Lecture

if04 把 BDC (Batch Data Communication / Batch Input) 定位成「模擬畫面輸入」的資料轉換技術——這題實際動手把它組出來。

⚠ 先講清楚這題的邊界：BDC 程式的核心資料是 **BDCDATA** 內部表，裡面每一列記錄「在哪個畫面 (**PROGRAM** + **DYNPRO**) 的哪個欄位 (**FNAM**) 打了什麼值 (**FVAL**)」，這份資料只能透過 **SHDB** 交易碼實際錄製一次畫面操作才能準確產生——**SHDB** 是互動式、逐格點選輸入的 GUI 操作，不是能用 API / MCP 工具自動化的東西 (跟本課程已經記錄過的 Smartform Form Painter、SM59/DBCO 設定畫面屬於同一類「GUI-only」限制)。這題能做到、也應該做到的是：把 **BDC** 程式裡「跟具體畫面欄位無關」的部分——呼叫骨架、兩種執行方式、錯誤訊息收集與轉譯——做成一支正確、通用、可以直接編譯執行的 **Z** 類別，具體某個交易碼的 **BDCDATA** 內容則留給有 SAP GUI 的人錄製後填入。

BDCDATA 結構 (**sap_get_source** 讀 DDIC 結構得到的真實欄位)：

欄位	型別	意義
program	BDC_PROG	這一步操作所在的畫面所屬程式 (通常是模組池，如 SAPMV45A)
dynpro	BDC_DYNR	畫面 (Dynpro) 號碼
dynbegin	BDC_START	'X' 表示這一系列是「換到新畫面」的起始列
fnam	FNAM____4	欄位名稱 (' BDC_OKCODE ' 這個特殊欄位用來塞功能碼 / Enter/下一步)
fval	BDC_FVAL	要填入的值

一支典型的 BDC 資料，結構長這樣 (**SHDB** 錄製完會自動產生類似的骨架，這裡示範語意，不是特定交易碼的真實內容)：

```
DATA: lt_bdcdata TYPE TABLE OF bdcdata,
      ls_bdcdata TYPE bdcdata.

" 第一個畫面
ls_bdcdata-program = 'SAPMV45A'.
ls_bdcdata-dynpro  = '0101'.
ls_bdcdata-dynbegin = 'X'.
APPEND ls_bdcdata TO lt_bdcdata.
CLEAR ls_bdcdata.

ls_bdcdata-fnam = 'VBAK-AUART'.
ls_bdcdata-fval = 'OR'.
APPEND ls_bdcdata TO lt_bdcdata.
CLEAR ls_bdcdata.

ls_bdcdata-fnam = 'BDC_OKCODE'.
```

```
ls_bdcdata-fval = '/00'.          " Enter
APPEND ls_bdcdata TO lt_bdcdata.
CLEAR ls_bdcdata.
```

" 下一個畫面繼續同樣的模式 (DYNBEGIN='X' 開一新畫面) ...

兩種執行方式：

① **Call Transaction Method**——直接、同步：

```
CALL TRANSACTION iv_tcode
  USING          it_bdcdata
  MODE          iv_mode          " 'A' 全程顯示畫面 / 'N' 全程不顯示 ( 背景速度最快 ) / 'E' 只在
出錯時顯示
  UPDATE        iv_update        " 'S' 同步 Update / 'A' 非同步 Update ( 呼應 if03 教過的
WAIT 概念——這裡是另一個場景會遇到同樣的同步/非同步取捨 )
  MESSAGES      INTO it_messages.
```

呼叫完立刻知道結果 (**sy-subrc** / **it_messages**)，適合「筆數不多、需要馬上知道成敗」的情境。

② **Session Method**——間接、非同步，建 Batch Input Session 讓 SM35 背景處理：

```
FUNCTION BDC_OPEN_GROUP
  IMPORTING
    VALUE(CLIENT) LIKE APQI-MANDANT DEFAULT SY-MANDT
    VALUE(GROUP) LIKE APQI-GROUPID DEFAULT FILLER12
    VALUE(USER) LIKE APQI-USERID DEFAULT FILLER12
    VALUE(KEEP) LIKE APQI-QERASE DEFAULT FILLER1
    ...
  EXPORTING
    VALUE(QID) LIKE APQI-QID
  EXCEPTIONS
    CLIENT_INVALID GROUP_INVALID GROUP_IS_LOCKED ... .
```

```
FUNCTION BDC_INSERT
  IMPORTING
    VALUE(TCODE) LIKE TSTC-TCODE DEFAULT FILLER4
  TABLES
    DYNPROTAB LIKE BDCDATA
  EXCEPTIONS
    INTERNAL_ERROR NOT_OPEN QUEUE_ERROR TCODE_INVALID ... .
```

```
FUNCTION BDC_CLOSE_GROUP
  EXCEPTIONS
    NOT_OPEN QUEUE_ERROR.
```

(三支都用 `sap_get_source` 直接讀出真實介面，屬於 Function Group `SBDC` / 套件 `SBDC` 。) 流程是：
`BDC_OPEN_GROUP` 開一個 Session → 迴圈對每一筆資料呼叫 `BDC_INSERT` (每次帶一組 `BDCDATA`，可以是不同交易碼) → `BDC_CLOSE_GROUP` 收尾。這支程式執行完只代表「Session 已經排好」，**還沒有真的執行**，要等人 (或排程) 進 SM35 手動或自動處理這個 Session，錯誤是在處理 Session 的當下才會冒出來，不是插入 Session 的當下——這是跟 Call Transaction Method 最大的行為差異：**Call Transaction 是同步立即跑**，**Session Method 是先排隊、之後才真正跑**，適合資料量大、不急著馬上知道結果、不想佔用前景作業時間的情境。

錯誤訊息收集與轉譯：不管哪種方式，收集到的訊息都是 `BDCMSGCOLL` 結構 (Call Transaction 的 `MESSAGES INTO`，或 Session 處理完後從 `SM35/APQD` 系列表查詢)：

欄位	型別	意義
<code>tcode</code>	<code>BDC_TCODE</code>	交易碼
<code>dynname</code>	<code>BDC_MODULE</code>	畫面所屬程式
<code>dynumb</code>	<code>BDC_DYNNR</code>	畫面號碼
<code>msgtyp</code>	<code>BDC_MART</code>	訊息類型 (E/W/S/I/A)
<code>msgid</code>	<code>BDC_MID</code>	訊息類別 (對應 <code>T100-ARBGB</code>)
<code>msgnr</code>	<code>BDC_MNR</code>	訊息號碼
<code>msgv1~msgv4</code>	<code>BDC_VTEXT1</code>	訊息變數 (對應 <code>&1~&4</code>)
<code>fldname</code>	<code>FNAM_____4</code>	出錯的欄位名稱

這幾個欄位組起來剛好就是 `FORMAT_MESSAGE` 這支標準 FM 要的輸入 (也是真實查證的介面)：

```
FUNCTION FORMAT_MESSAGE
IMPORTING
  VALUE(ID)   TYPE ANY DEFAULT SY-MSGID
  VALUE(LANG) TYPE ANY DEFAULT '-D'
  VALUE(NO)   TYPE ANY DEFAULT SY-MSGNO
  VALUE(V1)   TYPE ANY DEFAULT SY-MSGV1
  VALUE(V2)   TYPE ANY DEFAULT SY-MSGV2
  VALUE(V3)   TYPE ANY DEFAULT SY-MSGV3
  VALUE(V4)   TYPE ANY DEFAULT SY-MSGV4
EXPORTING
  VALUE(MSG) TYPE ANY
EXCEPTIONS
  NOT_FOUND.
```

把 `BDCMSGCOLL` 的 `msgid/msgnr/msgv1~msgv4` 餵給 `FORMAT_MESSAGE` 的 `ID/NO/V1~V4`，就能拿到一句組好變數的可讀文字 (例如「Field XXX 不可為空」而不是一串代碼)，這是比自己手動 `MESSAGE ID ... TYPE ... NUMBER ... WITH` 更省事的做法 (結果等價，`FORMAT_MESSAGE` 只是把同一件事包成一支可以在迴圈裡對很多筆訊息重複呼叫的 FM)。

學習目標

- 認識 **BDCDATA** / **BDCMSGCOLL** 兩張結構的真實欄位，理解一支 BDC 程式的資料本質是「畫面 + 欄位 + 值」的清單
- 能寫出 Call Transaction Method 跟 Session Method 兩種呼叫骨架，並講出彼此的行為差異（同步立即 vs 先排隊後處理）
- 能用 **FORMAT_MESSAGE** 把 **BDCMSGCOLL** 轉成可讀文字
- 認清「**SHDB** 錄製」是這整套技術裡唯一沒辦法自動化、必須有 SAP GUI 才能做的步驟

事前準備

不需要既有物件；這題會新建一支**通用、不綁定任何特定交易碼**的 Z 類別 **ZCL_IF05_BDC_RUNNER**，把 Lecture 教的骨架包裝成可重用的方法。

題目需求

1. 設計並實作 **ZCL_IF05_BDC_RUNNER**，至少包含：
 - **run_via_call_transaction**：包一層 **CALL TRANSACTION ... USING ... MODE ... UPDATE ... MESSAGES INTO**
 - **run_via_session**：包一層 **BDC_OPEN_GROUP** → 迴圈 **BDC_INSERT** → **BDC_CLOSE_GROUP**
 - **format_messages**：把一張 **BDCMSGCOLL** 表轉成一張可讀文字的 **string_table**，內部呼叫 **FORMAT_MESSAGE**
2. **format_messages** 要附 ABAP Unit 測試——這個方法是純邏輯轉換（輸入一張假的 **BDCMSGCOLL** 表、輸出可讀文字），完全不需要真的執行過 BDC 或連到任何畫面，可以直接用假資料測試，呼應課程慣例「純邏輯的方法要寫測試」
3. **run_via_call_transaction** / **run_via_session** 這兩個方法不寫測試（會真的觸發交易執行 / 建立 Session，依課程慣例「會碰資料庫 / 外部畫面的方法」不寫 ABAP Unit）

答案

見 **zcl_if05_bdc_runner.clas.abap**（含 **zcl_if05_bdc_runner.clas.testclasses.abap**，SAP 端物件 **ZCL_IF05_BDC_RUNNER**，套件 **\$TMP**）。已在系統上實際建立、啟用、**sap_run_unit_test** 全數通過（3/3）。

團隊實務備註

- **BDC_INSERT** / **BDC_OPEN_GROUP** / **BDC_CLOSE_GROUP** 三支介面已用 **sap-adt MCP** 於 client 130 查證（Function Group **SBDC**，套件 **SBDC**），**FORMAT_MESSAGE** 同樣查證存在（Function Group **SP00**，套件 **SP00**——這支雖然名字聽起來像列印相關，實際是通用的訊息格式化工具，套件歸屬只是歷史因素，不用太在意）。
- 這題沒有針對任何真實交易碼做端對端測試（沒有真的呼叫 **run_via_call_transaction** 對一個交易碼跑一次）——這需要一份真實的 **BDCDATA**，而那份資料只能靠 **SHDB** 錄製取得。如果之後要驗證這支類別能不能真的用，需要你在 SAP GUI 對一個安全的目標交易碼（建議用課程自己的 Z 交易，例如 **ex21** 的 **ZTR21_CLASS** **SM30** 維護畫面，資料範圍完全在自己控制內）錄一次 **SHDB**，把錄出來的 **BDCDATA** 貼給我，我可以幫忙組進呼叫程式裡實際跑一次驗證。
- **format_messages** 第一版有一個真實踩到的 **bug**，過程本身就是很好的教材：一開始的寫法是「呼叫 **FORMAT_MESSAGE** 之前先準備一句 fallback 文字，指望找不到訊息時這句 fallback 能保留下來」——結

果 ABAP Unit 測試（刻意用一個確定不存在的訊息類別 ZZ 驗證）直接抓到：FORMAT_MESSAGE 找不到訊息、RAISE NOT_FOUND 提前結束時，EXPORTING VALUE(MSG) 這個傳值參數還是把呼叫端的變數蓋成空白，並沒有維持呼叫前的原值——這跟直覺相反（很多人以為例外路徑不會動到 EXPORTING 參數）。正確做法是呼叫完用 sy-subrc 明確判斷，成功才採用輸出值。這個坑已經記進 .claude/rules/sap-adt-mcp.md 第 18 節。

- 另外兩個順手踩到、也記進第 18 節的坑：CALL FUNCTION 不支援 IMPORTING xxx = DATA(lv_yyy) 這種 inline 宣告（只有 Method 呼叫支援，寫了會在啟用時報兩則訊息）；驗證「訊息類別 ZZ 真的不存在」時改用 /sap/bc/adt/datapreview/freestyle 端點直接下帶 WHERE 條件的查詢（比只能整表撈的 datapreview/ddic 端點好用很多），順便意外發現訊息類別 00 的號碼 999 在這套系統真的有資料（Table extension for compression successful），推翻了「隨便挑個大號碼應該不存在」的猜測。

思考題

1. run_via_session 呼叫完 BDC_CLOSE_GROUP 後，程式的 sy-subrc 是 0，這代表「資料已經成功寫進 SAP」了嗎？（提示：不代表——sy-subrc = 0 只代表 Session 建立成功，資料要等 SM35 處理這個 Session 時才會真的跑過交易畫面；這跟 if02/if03 學到的「呼叫成功不代表資料庫已經真的寫入」是同一種「非同步」概念的另一種展現）
2. MODE = 'N'（全程不顯示畫面）通常比 MODE = 'A'（全程顯示畫面）快很多，那為什麼開發階段常常故意用 MODE = 'A' 而不是一開始就用 'N'？（提示：MODE = 'A' 會讓畫面真的跳出來，可以親眼看到 BDC 在哪一步卡住、哪個欄位值不對，除錯效率更高；等確認整組 BDCDATA 沒問題了，正式上線 / 大量執行時才切回 'N' 追求速度）
3. 如果同一支程式，未來要改成呼叫一個「已經有 BAPI」的交易，該怎麼決定要不要把 BDC 換成 BAPI？（提示：直接呼應 if04 的思考題 2——原則上優先選 BAPI，BDC 保留給「沒有 BAPI、沒有 Direct Input」的情境）